MEDICENT V3 • SISTEMA DE GESTIÓN Y ADHERENCIA FARMACOLÓGICA

Guía de Integración y Contexto para Desarrollo Mobile

Fecha: Octubre 2026
Versión Backend: V3.0.0 (Flask RESTful + JWT)

API COMPLETA • 100% OPERATIVA

> **Propósito de este Documento**
> Este documento técnico proporciona a los desarrolladores de la aplicación móvil (Flutter / React Native / Android nativo) todo el contexto del dominio médico, el diagrama de base de datos, los contratos de endpoints REST, los esquemas de autenticación JWT y las directrices para estructurar la nueva carpeta de trabajo mobile.

1. Resumen Ejecutivo del Backend

El Backend de **Medicent V3** ha completado su arquitectura multicapa (Rutas → Controladores → Servicios → Modelos ORM) y cuenta con una suite automatizada de **28 pruebas continuas aprobadas** (Unitarias y de Integración con SQLite en memoria y MySQL para producción).

La aplicación está construida para resolver el problema de **falta de adherencia terapéutica** en pacientes ambulatorios y facilitar la labor de supervisión de sus cuidadores asignados.

2. Modelo de Dominio y Base de Datos Relacional

La base de datos relacional (MySQL `medicent_v3`) se compone de **5 entidades esenciales**:

- **`usuarios`**: Almacena credenciales seguras (hash Bcrypt), nombres y roles permitidos (`paciente`, `cuidador`, `admin`).
- **`pacientes_cuidadores`**: Tabla asociativa muchos a muchos que vincula qué cuidador tiene bajo su cargo a qué paciente.
- **`medicamentos`**: Catálogo de fármacos con nombre, descripción y forma farmacéutica (tableta, jarabe, inyectable, etc.).
- **`tratamientos`**: Prescripción médica que asocia un paciente con un medicamento, dosis, frecuencia horaria, rango de fechas (inicio y fin) y hora de la primera toma.
- **`registros_tomas`**: Cronograma individualizado de tomas generado automáticamente con estados: `pendiente`, `tomada`, `omitida`, `retrasada`.

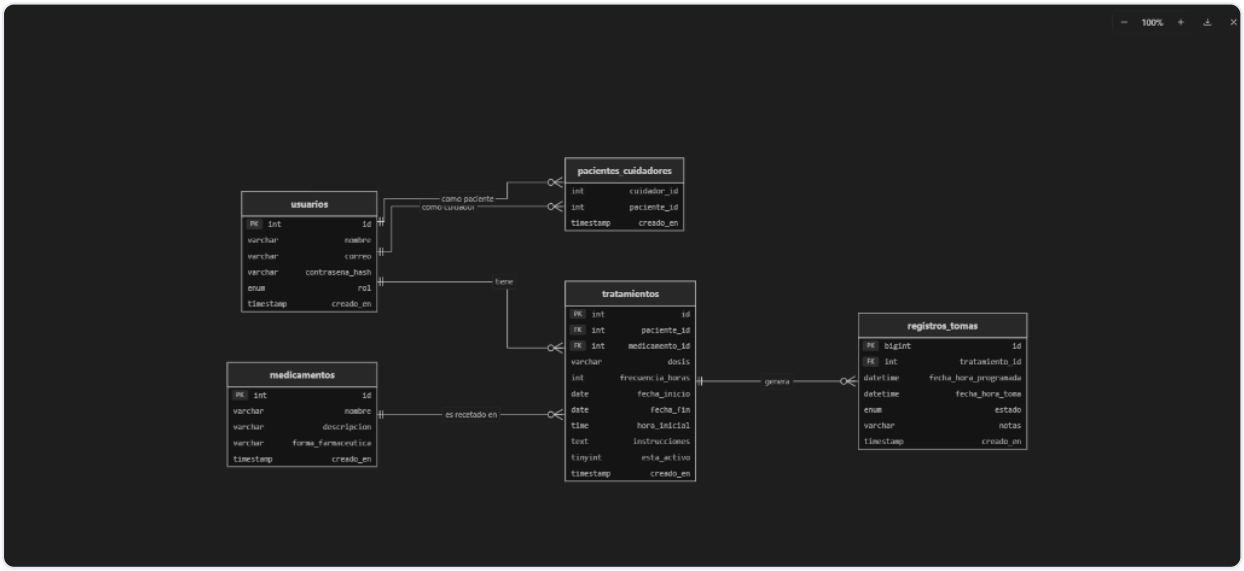


Figura 1: Diagrama Entidad-Relación del Backend Medicent V3

3. Especificación de Endpoints y Contratos de la API REST

El backend expone endpoints en formato JSON. En desarrollo local la URL base estándar es: `http://127.0.0.1:5000/api` (o `http://10.0.2.2:5000/api` para emuladores Android).

A. Módulo de Autenticación y Autorización (`/api/auth`)

Método	Ruta	Seguridad	Descripción y Carga Útil (Payload)	Retorno
POST	`/api/auth/registro`	Pública	Registra un nuevo usuario. `{"nombre": "Carlos", "correo": "carlos@med.com", "contrasena": "123456", "rol": "paciente"}`	`201 Created`
POST	`/api/auth/login`	Pública	Inicia sesión y genera token JWT. `{"correo": "carlos@med.com", "contrasena": "123456"}` *Retorna: token, id, nombre, rol.*	`200 OK` `401 Error`
POST	`/api/auth/asignar-paciente`	JWT (Cuidador/Admin)	Vincula un paciente a un cuidador responsable. `{"cuidador_id": 2, "paciente_id": 1}`	`201 Created` `403 Prohibido`

> **Uso del Token JWT en Mobile**
>
> En las rutas protegidas, la app móvil debe enviar el encabezado HTTP:
>
> ```
> Authorization: Bearer <tu_jwt_token>
> ```

B. Módulo de Medicamentos (`/api/medicamentos`)

Método	Ruta	Seguridad	Descripción y Carga Útil	Retorno
GET	`/api/medicamentos`	Pública / App	Lista todo el catálogo de medicamentos registrados para selección rápida en formularios.	`200 OK` (Array)
POST	`/api/medicamentos`	Pública / App	Registra un nuevo medicamento. `{"nombre": "Ibuprofeno", "descripcion": "Antiinflamatorio", "forma_farmaceutica": "Tableta 400mg"}`	`201 Created`

C. Módulo de Tratamientos Médicos (`/api/tratamientos`)

Método	Ruta	Seguridad	Descripción y Carga Útil	Retorno
POST	`/api/tratamientos`	App	Crea un tratamiento y **calcula automáticamente el calendario de tomas**. `{"paciente_id": 1, "medicamento_id": 2, "dosis": "1 cápsula", "frecuencia_horas": 8, "fecha_inicio": "2026-11-01", "fecha_fin": "2026-11-03", "hora_inicial": "08:00:00"}`	`201 Created`
GET	`/api/tratamientos/paciente/<id>`	App	Devuelve la lista de tratamientos activos y pasados correspondientes al paciente indicado.	`200 OK` (Array)

D. Módulo de Tomas y Notificaciones (`/api/tomas`)

Método	Ruta	Seguridad	Descripción y Carga Útil	Retorno
GET	`/api/tomas/paciente/<id>`	App	Obtiene el cronograma de tomas con fechas/horas y estado. Es el endpoint principal para el calendario y alarmas de la app móvil.	`200 OK` (Array)
PATCH	`/api/tomas/<toma_id>`	App	Permite al paciente o cuidador confirmar una toma. `{"estado": "tomada"}` → Actualiza estado y guarda `fecha_hora_toma` con la hora actual.	`200 OK`

4. Estructura Recomendada para el Proyecto Mobile

Para mantener un desacoplamiento limpio del repositorio backend, se recomienda crear una carpeta hermana en el espacio de trabajo (por ejemplo `Mobile_Medicent_v3`). A continuación se ilustra la arquitectura de código recomendada (Clean Architecture / BLoC o Providers):

```
Mobile_Medicent_v3/
├── android/ / ios/            # Plataformas nativas
├── assets/                    # Iconos médicos, logos, sonidos de alarma
│   ├── icons/
│   └── images/
├── lib/ (o src/)
│   ├── core/                  # Utilitarios y configuración global
│   │   ├── api/
│   │   │   ├── api_client.dart       # Cliente HTTP (Dio / Axios / Fetch) con interceptor JWT
│   │   │   └── api_endpoints.dart    # Constantes con URLs de los endpoints
│   │   ├── theme/                    # Colores accesibles, tipografía grande para adultos mayores
│   │   └── utils/                    # Parseo de fechas ISO 8601 a hora local
│   ├── models/                # Modelos serializables que reflejan el backend
│   │   ├── usuario_model.dart        # {id, nombre, correo, rol}
│   │   ├── medicamento_model.dart    # {id, nombre, forma_farmaceutica}
│   │   ├── tratamiento_model.dart    # {id, paciente_id, dosis, frecuencia}
│   │   └── toma_model.dart           # {id, fecha_hora_programada, estado}
│   ├── services/              # Lógica de consumo de API y almacenamiento local
│   │   ├── auth_service.dart         # Login, registro y guardado seguro de token (SecureStorage)
│   │   ├── notification_service.dart # Notificaciones locales (alarmas a la hora exacta de la toma)
│   │   └── tratamiento_service.dart  # Consulta y marcado de tomas
│   ├── screens/ / views/      # Pantallas de la interfaz de usuario
│   │   ├── auth/                     # Login y Registro
│   │   ├── home/                     # Dashboard diferenciado según rol (Paciente vs Cuidador)
│   │   ├── tomas/                    # Lista del día con botones grandes [Marcar Tomada]
│   │   ├── tratamientos/             # Formulario de prescripción o consulta
│   │   └── perfil/                   # Datos personales y cerrar sesión
│   └── main.dart                     # Punto de entrada
└── README.md
```

5. Flujos Clave de la Interfaz Móvil

Flujo 1: Notificación y Marcado de Toma (Experiencia Paciente)

1. El paciente inicia sesión; la app almacena su `token` JWT y su `id` de paciente de forma segura.
2. La app invoca `GET /api/tomas/paciente/{id}` y programa recordatorios locales en el dispositivo móvil con el servicio de alarmas nativo.
3. Al sonar la alarma, se muestra una notificación interactiva con botones: "*Tomar Ahora*" o "*Posponer 15 min*".
4. Al pulsar "*Tomar Ahora*", la app hace un `PATCH /api/tomas/{toma_id}` con `{"estado": "tomada"}`, actualizando el backend en tiempo real.

Flujo 2: Supervisión Remota (Experiencia Cuidador)

1. El cuidador inicia sesión (recibe `rol: "cuidador"`).
2. La app muestra la lista de pacientes a su cargo vinculados a través de `pacientes_cuidadores`.
3. Al seleccionar un paciente, consulta su calendario de tomas y alerta en color rojo las tomas que sigan en estado `pendiente` habiendo superado la hora límite programada.

6. Parámetros Técnicos de Conectividad

Entorno de Prueba Mobile	Host / Dirección Base	Observación Técnica
Emulador Android Oficial	`http://10.0.2.2:5000/api`	El alias `10.0.2.2` mapea automáticamente al `localhost` de la máquina anfitriona.
Simulador iOS	`http://127.0.0.1:5000/api`	Comparte la misma interfaz de red loopback que la máquina.
Dispositivo Físico (WiFi)	`http://<IP_LOCAL_PC>:5000/api`	Ambos deben estar en la misma red WiFi y Flask debe iniciarse con `flask run --host=0.0.0.0`.